Mantenimiento del simulador de torneo

Actividad 2, Unidad 2

Manuel Lozano

17 de octubre de 2026

1. Introducción y selección del código

El caso de clase es ConstruYA, una cadena de ferreterías cuyo sistema acumula deuda técnica. El componente intervenido es de autoría propia, escrito hace unos tres años: un simulador de torneo de fútbol en C++. Arma ocho grupos de cuatro equipos y luego octavos, cuartos, semifinales, tercer puesto y final. Lee `equipos2.txt` (nombre y probabilidad de gol por línea).

El original, `main2.cpp`, tenía 300 líneas, por encima del mínimo de 150. Sirve para el ejercicio porque concentra método largo, duplicación, números mágicos, estado oculto y responsabilidades mezcladas, y porque su salida se puede fijar con una semilla. El proceso fue diagnóstico, pruebas de caracterización, refactorización en una rama, validación y merge a `main`.

2. Diagnóstico

cppcheck 2.21 (`-enable=all`) no detecta método largo, duplicación ni números mágicos. Sólo avisó de campos de `Equipo` sin inicializar y de una referencia que podía ser `const`. Esos smells se sostienen con clang-tidy 23 y con lizard 1.24, este último con umbral de complejidad mayor que 10 o longitud mayor que 50. El umbral de fábrica de lizard no marca el archivo.

Smell	Ubicación	Evidencia	Técnica
Método largo	`SimularPartido`, 65–163	lizard: CCN 15, longitud 99. clang-tidy: complejidad cognitiva 31	Extraer método
Duplicación	bucles de 90 y 30 min; `rand` seis veces	sin detector de clones; clang-tidy marca `rand`	`SimularTiempo` y aleatorio único
Números mágicos	90, 30, 5, 8, 2006, 31, 28	`readability-magic-numbers`	`constexpr`
Estado oculto	`ObtenerFecha`, `static`	inspección; las pruebas dependen del orden	se documenta; no se alteró la secuencia
Partido dentro de `Equipo`	`estadio`, `fecha`	inspección; cppcheck ve los campos sin inicializar	estructura `Partido`

3. Refactorización

Cinco técnicas, un commit cada una, en `refactor/mantenimiento-simulador`. Los valores y el orden de `std::rand` no cambiaron. `std::rand` y `mt19937` no se unieron: son algoritmos distintos y la salida con semilla 1 dejaría de coincidir.

Antes, el minuto de juego estaba copiado para el tiempo reglamentario y para la prórroga:

```
for (int minuto = 0; minuto < 90; minuto++) {
    double resultadoAleatorio =
        static_cast<double>(std::rand()) / RAND_MAX;
    if (resultadoAleatorio < probabilidadGolEquipo1 / 90)
        golesEquipo1++;
    resultadoAleatorio =
        static_cast<double>(std::rand()) / RAND_MAX;
    if (resultadoAleatorio < probabilidadGolEquipo2 / 90)
        golesEquipo2++;
}
```

Después, ambos tramos llaman a la misma función, y el número 90 es `kMinutosReglamento`:

```
void SimularTiempo(int minutos, double probabilidad1,
        double probabilidad2, int& goles1, int& goles2) {
    for (int minuto = 0; minuto < minutos; minuto++) {
        if (AleatorioUnitario() < probabilidad1 / minutos)
            goles1++;
        if (AleatorioUnitario() < probabilidad2 / minutos)
            goles2++;
    }
}
```

También se extrajeron la tanda de penales, la actualización de estadísticas y la impresión. Las variables de `main` pasaron a `snake_case`. Fecha y estadio salieron de `Equipo` hacia `Partido`, y el código quedó en `equipo.h`, `simulador.h`, `simulador.cpp` y `main.cpp`. La segunda petición de estadio, que el original hacía y no imprimía, se conservó para no desplazar los estadios siguientes.

4. Herramientas de análisis

La estática es cppcheck, complementada con clang-tidy y lizard. La dinámica prevista era AddressSanitizer y UBSan. En MSYS2 UCRT64 el enlace falla (`cannot find -lasan` y `-lubsan`); Valgrind no existe en Windows. La evidencia está en `evidencia/sanitizers_antes.txt`. La herramienta dinámica usada es gcov con gcovr, porque instrumenta el binario y mide la ejecución real de las pruebas.

Métrica	Antes	Después
`SimularPartido`	66 líneas, CCN 15, longitud 99	20 líneas, CCN 4, longitud 26
`main`	66 líneas, CCN 12, longitud 100	66 líneas, CCN 12, longitud 82
CCN media	5,2; dos funciones sobre el umbral	3,4; sólo `main` sigue encima
cppcheck	4 campos sin inicializar y una referencia	4 campos sin inicializar
Líneas cubiertas	144/203 = 70 %	146/149 = 98 %
Ramas cubiertas	170/418 = 40 %	135/216 = 62 %

La cobertura sube, en parte, porque desaparecieron ramas duplicadas (418 a 216), no porque el torneo recorra caminos nuevos. `main` conserva CCN 12: separar el archivo no partió esa función.

5. Control de versiones

Repositorio público: https://github.com/Manulol1904/simulador-torneo. Rama principal `main`. La refactorización vivió en `refactor/mantenimiento-simulador`, con commits atómicos (`refactor: reemplazar números mágicos...`, `refactor: extraer SimularTiempo...`, entre otros). La integración fue un merge `-no-ff`, commit `4a4dd66`, mensaje `merge: integrar refactorización del simulador`, con dos padres. El tag `v1.0-refactor` apunta a ese merge. El grafo está en `evidencia/git_log_graph.txt`.

6. Pruebas y validación

GoogleTest 1.15.2. Diez pruebas, todas en verde antes y después: diferencia de goles, victoria, derrota y empate, penales que siguen empatados a cero, formato de fecha `dd/mm/2006`, desempate de grupos y ganadores de ronda por goles acumulados. El golden master compara la salida completa con semilla 1 contra `evidencia/salida_original.txt`. Después de refactorizar, los dos archivos tienen 7590 bytes y el mismo SHA-256. `evidencia/diff_golden_master.txt` registra `IDENTICAS=si`.

7. Bugs detectados y trabajo futuro

No se corrigieron dentro de la refactorización. Quedan para una rama `fix/` con pruebas nuevas: `ObtenerFecha` solo cambia de mes en enero y febrero; el ganador de una llave se decide por goles acumulados del torneo; el tercer puesto usa dos ganadores de cuartos y no a los perdedores de las semifinales; los penales se suman al marcador y pueden empatar; las estadísticas de grupos se arrastran a la eliminatoria; `iss » nombre` corta los nombres con espacio. `main` sigue siendo la función más compleja.

8. Reflexión

El cambio reduce el costo de un arreglo local. Un ajuste de la prórroga o de los penales ya no obliga a leer un solo bloque de cien líneas, y el golden master avisa si la salida del torneo se mueve. Eso baja el riesgo de una falla nueva al mantener el código. El límite es el que las pruebas fijan el comportamiento actual, incluidos los bugs: una mejora futura, por ejemplo cobrar la inscripción con una pasarela de pagos, no debería colarse en `SimularPartido`. Tendría un módulo propio y pruebas propias. Hoy `main` y la fecha `static` siguen acoplando el calendario al orden de las llamadas, así que ese cambio todavía exige cuidado. La sostenibilidad mejoró en la simulación del partido; la deuda que queda está localizada y escrita.

9. Contraste con la rúbrica

Criterio	Dónde se evidencia
C1 Diagnóstico	Sección 2 y `evidencia/diagnostico_smells.txt`. Cinco smells con cppcheck, clang-tidy y lizard.
C2 Refactorización	Sección 3. Cinco técnicas, antes y después. El golden master no cambió.
C3 Herramientas	Sección 4. Estática: cppcheck. Dinámica: gcov/gcovr, porque ASan no enlaza en este MSYS2.
C4 Git	Sección 5. Rama `refactor/mantenimiento-simulador`, commits atómicos, merge `4a4dd66` y tag `v1.0-refactor`.
C5 Pruebas	Sección 6. Diez pruebas en verde y diff vacío del golden master.